

FPGA-Accelerated HFT Edge Device (Research Project)

Aneesh Poddutur

*Electrical and Computer Engineering
Purdue University
West Lafayette, United States
apoddutu@purdue.edu*

Dheeraj Krishna Chaitanya Karanam

*Electrical and Computer Engineering
Purdue University
West Lafayette, United States
dkarana@purdue.edu*

Bharath Chandar Natarajan

*Mechanical Engineering
Purdue University
West Lafayette, United States
bnataraj@purdue.edu*

Shraddha Sahoo

*Electrical and Computer Engineering
Purdue University
West Lafayette, United States
sahoo15@purdue.edu*

Yiyun Wang

*Electrical and Computer Engineering
Purdue University
West Lafayette, United States
wang4944@purdue.edu*

Abstract—This document serves as the Final Project Report for an FPGA-Accelerated High-Frequency Trading (HFT) Edge Device. We demonstrate a 600x-1000x speedup of Black-Scholes computation, with an introduction of of roughly 2-3% error. We provide current state-of-the-art approaches, how our work meshes with the existing HFT landscape, and a design of a potential edge HFT device.

Index Terms—Edge Device, FPGA, High Frequency Trading

I. INTRODUCTION

High-Frequency Trading (HFT) is a form of algorithmic trading that leverages high-throughput, low-latency computing to execute numerous financial transactions at extremely high speeds, capitalizing on small price movements such as commodity price mismatches or options pricing recalculations. The success of HFT depends on ultra-low latency, which is often compromised by the geographical distance between trading systems and exchange servers. Traditional HFT infrastructure, reliant on costly hardware like Application-Specific Integrated Circuits (ASICs) and direct Ethernet connections, is largely inaccessible to individual traders and imposes significant costs on large firms. This research proposes a portable, low-power edge device integrating an ESP32 microcontroller with a Field-Programmable Gate Array (FPGA) to enable low-latency trading computations closer to exchanges, offering a cost-effective and accessible solution for both institutional and individual traders.

A. Target Problem

The core challenge in HFT is achieving ultra-low latency in processing market data and executing trades, particularly when trading systems are located far from financial exchanges. Physical distance increases cable lengths, introducing transmission delays that can undermine profitability in a domain where nanosecond differences are critical. Not all traders, especially individual or smaller entities, can establish infrastructure near exchanges due to space, cost, or logistical constraints. This

project aims to develop a portable edge device using an FPGA and an ESP32 microcontroller to minimize latency by enabling market data processing and trading algorithm execution closer to exchange servers, while maintaining affordability and ease of deployment.

B. Motivation

The motivation for this research is to reduce latency in HFT while making it more accessible to a diverse range of users, from large firms like Citadel Securities to individual traders like a student named Jeremy. Traditional HFT systems rely on direct Ethernet connections, high-speed interconnects like PCIe, and specialized hardware such as FPGAs or ASICs, which are expensive and require proximity to exchanges. Distant trading locations exacerbate latency due to longer cable lengths, putting remote traders at a disadvantage. A compact, portable edge device offers a solution by enabling proximity to exchanges without the need for costly infrastructure, democratizing access to HFT and reducing energy consumption for scalable deployments.

C. Potential Applications

The proposed edge device offers several applications across the financial sector:

- 1) **Institutional Trading:** Large firms can deploy edge devices near exchanges for preliminary market data processing, reducing latency and enhancing HFT efficiency.
- 2) **Individual Trading:** The device's affordability and portability enable individual traders, such as students in dorm rooms, to implement low-volume HFT strategies without significant infrastructure.
- 3) **Educational Use:** Academic researchers and students can use the device as a cost-effective platform for experimenting with trading algorithms and hardware acceleration.

- 4) **Scalable Deployment:** Low power consumption and wireless capabilities allow deployment in diverse settings, such as remote locations or temporary setups near exchanges.
- 5) **Cross-Market Arbitrage:** The device can exploit price mismatches across exchanges by executing rapid computations, such as the Black-Scholes equation for options pricing.

D. Existing Work and Limitations

Current HFT systems combine advanced algorithms and specialized hardware to achieve ultra-low latency. Major firms like Citadel Securities, Virtu Financial, and Jump Trading use algorithms such as Reinforcement Learning (RL), Deep Learning (DL), Adaptive Market-Making Algorithms (AMM), and Natural Language Processing (NLP), trained on extensive historical and real-time market data. Hardware solutions include FPGAs, ASICs, or GPU/CPU accelerators. ASICs offer the highest speed and lowest power consumption but are costly and require 1–2 years for development. FPGAs provide high speed, customization, and lower cost, while GPUs/CPUs are general-purpose but slower. High-speed interconnects like PCIe and direct Ethernet connections to exchanges further minimize latency.

Existing HFT systems face several limitations:

- 1) **Geographical Constraints:** The need for proximity to exchanges limits scalability, as establishing infrastructure near servers is often infeasible due to space and cost.
- 2) **High Costs:** ASICs are prohibitively expensive and time-consuming to develop, viable only for large firms. FPGA-based systems, while cheaper, can still be costly with high-speed interconnects.
- 3) **Accessibility Barriers:** Expensive hardware and infrastructure exclude smaller firms and individual traders, limiting market competition and innovation.
- 4) **Power Consumption:** Systems using GPUs or CPUs consume significant power, posing challenges for large-scale deployments.
- 5) **Complexity:** Advanced algorithms like RL and DL require substantial computational resources and expertise, creating barriers for smaller entities.

This project addresses these limitations by proposing a cost-effective, low-power edge device combining an ESP32 microcontroller for wireless market data acquisition with an FPGA for hardware-accelerated trading computations. By enabling deployment closer to exchanges and reducing reliance on expensive infrastructure, the solution enhances performance, affordability, and accessibility in HFT.

We will be exploring the role of edge devices in High-Frequency Trading (HFT), and constructing one for use with an external financial API. Our research will involve setting up a rudimentary method of wirelessly sending market data to an ESP32 microcontroller (through HTTPS/TCP/UDP requests), and having the ESP32 accelerate computation through

an FPGA connected via a Serial Peripheral Interface (SPI) connection.

II. METHODOLOGY

The objective of this project is to accelerate High-Frequency Trading (HFT) computations by implementing the Black-Scholes option pricing model on FPGA hardware. The system aimed to integrate a Xilinx Artix-7 FPGA (Arty A7 board) and an ESP32 microcontroller, working together to provide a low-latency, deterministic computation pipeline and wireless data interfacing. This hardware-software co-design aimed to create a real-time, edge-deployed trading accelerator that reduces latency and enhances accessibility for diverse users. However, due to hardware and time constraints, this project will serve to implement Black-Scholes on the FPGA and measure the pure speedup achieved by the FPGA hardware.

A. Black-Scholes Equation

At the core of this system is the FPGA-based implementation of the Black-Scholes pricing model. The Black-Scholes formula proves that a call option for a European security can be deterministically calculated. It is given by:

$$C = N(d_1)S_t - N(d_2)Ke^{-rt}$$

$$\text{where } d_1 = \frac{\ln \frac{S_t}{K} + \left(r + \frac{\sigma^2}{2}\right)t}{\sigma\sqrt{t}}$$

$$d_2 = d_1 - \sigma\sqrt{t}$$

Where:

C	=	call option price
$N(\cdot)$	=	CDF of the standard normal distribution
S_t	=	spot price of the asset
K	=	strike price
r	=	risk-free interest rate
t	=	time to maturity
σ	=	volatility of the asset

B. System Architecture

The proposed system comprised of an ESP32 microcontroller and a Xilinx Artix-7 FPGA (Arty A7 board), interconnected via a Serial Peripheral Interface (SPI) bus. The ESP32 would handle wireless data reception and system coordination, fetching real-time market data or synthetic data via HTTPS/TCP/UDP requests from an external financial API (e.g., Alpha Vantage). The FPGA would perform hardware-accelerated computations, specifically implementing the Black-Scholes pricing model for European options. The ESP32 would also log processed data for benchmarking. Data would flow from the external API to the ESP32, through the SPI module to the FPGA for computation, and back to the ESP32 for logging or transmission to an edge server or database.

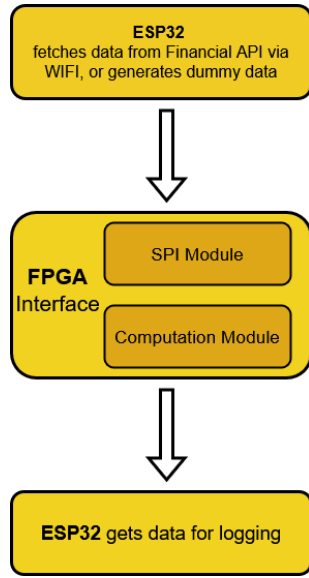


Fig. 1. Proposed System Architecture

As aforementioned, the SPI link and communication was deprioritized, and accelerating the digital design of the hardware and measuring speedup became the main objective of the project. Therefore, the SPI link was severed, and data was generated on both the FPGA and the microcontroller for benchmarking purposes.

C. Core Computation Strategy and FPGA Design

The primary objective of the FPGA design was to achieve rapid computation of the Black-Scholes option price while maintaining precision. Given the high-frequency trading (HFT) context, where even microseconds can impact profitability, the following strategies were employed:

- 1) **Fixed-Point Arithmetic (Q6.10 and Q4.12):** The Black-Scholes model was implemented entirely using fixed-point arithmetic with a Q6.10 format (6 integer bits, 10 fractional bits). This format offers a balanced trade-off between precision and computational efficiency, avoiding the overhead of floating-point operations.
- 2) **Mathematical Function Optimization:** The most computationally expensive functions in the Black-Scholes model (such as natural logarithm, exponential, and square root) were precomputed and stored in lookup tables (LUTs). This approach allows the FPGA to rapidly retrieve function values without performing complex calculations in real time.
- 3) **Parallel Computation and Resource Reuse:** The FPGA's minimal resources were taken into account, and mathematical operations were not replicated. In other words, each lookup table was instantiated only once, and reused in varying stages of the computation's finite state machine. The FSM also allowed for parallelized computation of some values.

The following figures showcase the design of the FPGA, including the finite state machine stages utilized for data computation, and the RTL modules designed for computation:

Data arrives at the FPGA,
start signal is pulsed for one clock cycle

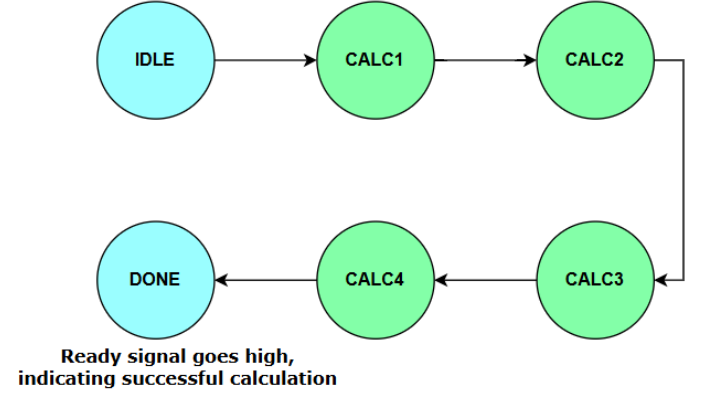


Fig. 2. FPGA Six-Stage Finite State Machine

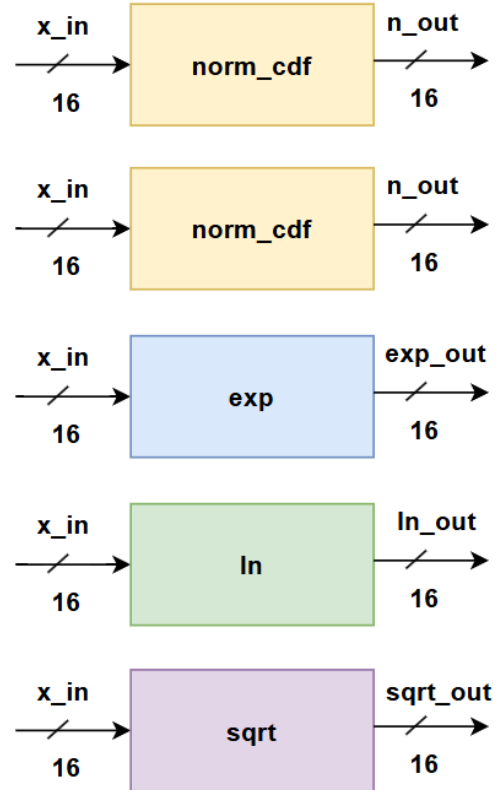


Fig. 3. Five Math Modules Designed for Computation

The *norm-cdf* modules both perform a single-cycle lookup, by utilizing a 512-entry lookup table formatted in Q4.12 fixed point. The *exp*, *ln*, and *sqrt* modules all perform a single-

cycle lookup, by utilizing a 512-entry lookup table formatted in Q6.10 fixed point. Division occurs only once per cycle stage, ensuring reuse of the divider logic. The subsystem integration's top module is show below:

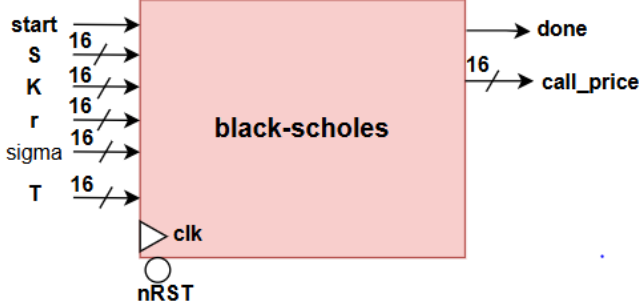


Fig. 4. Top Module

D. External API Integration

External API calls have been implemented for Alpha Vantage, Polygon, Financial Modeling Prep (FMP), FINNHUB, TIINGO, and Federal Reserve Economic Data (FRED) to fetch real-time and historical market data for the edge device. These APIs provide inputs such as stock prices, options data, economic indicators, and fundamental data, which are transmitted to the ESP32 via Wi-Fi. The integration supports diverse data sources to enhance the system's flexibility, enabling the device to process a wide range of financial information for HFT applications.

E. SPI Bus for Inter-Device Communication

It was intended for a Serial Peripheral Interface (SPI) bus to connect the FPGA and ESP32 for bidirectional communication:

- **ESP32 (SPI Master):** Sends market data and computation requests.
- **FPGA (SPI Slave):** Receives inputs, executes computations, and returns results.

The SPI protocol includes a simple framing structure with command identifiers, data length, and payload. This allows fast and structured data transfer, critical for low-latency performance.

F. Wireless Communication and Data Handling

The ESP32 module was intended to act as a communication gateway and controller. It would support two mode:

- 1) **Live Mode:** Real-time financial data is fetched from an external API (e.g., stock quotes or option chains) via Wi-Fi.
- 2) **Benchmark Mode:** Synthetic data is generated internally for testing and validation of system performance.

Upon receiving data, the ESP32 formats the inputs and transmits them over SPI to the FPGA. Once the FPGA returns the computed results, the ESP32 forwards them over Wi-Fi to an edge server, database, or local display interface.

This hardware-software co-design aimed to enable a real-time, edge-deployed trading accelerator. By isolating computation on FPGA and delegating control and communication to ESP32, the system would have achieved efficient task partitioning and a scalable framework for financial analytics.

III. EVALUATION

Experimental Setup

Hardware Configuration:

- **FPGA Board:** Digilent Arty A7-35T, based on Xilinx Artix-7 FPGA.
- **Microcontroller:** ESP32-WROOM-32

Software Tools:

- **Vivado Design Suite 2024.2:** Used for RTL synthesis, implementation, and bitstream generation.
- **ESP-IDF v5.1:** Used for ESP32 development, including SPI communication and Wi-Fi networking.
- **Python (NumPy, Matplotlib):** Used for the software baseline implementation of the Black-Scholes model and result visualization.

1) *Design:* The RTL design was created on AMD's Vivado platform. Both the behavioral simulation and the synthesis of the final design were performed utilizing Vivado.

The final design was able to be synthesized. The following FPGA Resources were utilized in the final synthesis run:

TABLE I
FPGA RESOURCE UTILIZATION

Resource	Count
Slice LUTs	4492
Slice Registers	141
F7 Muxes	153
F8 Muxes	38
DSPs	10
Bonded IOB	100
BUFGCTRL	1

The number of available LUTs available on our target FPGA board were a source of concern. However, by the final RTL design iteration, total LUT usage was only around 25%.

2) *Accuracy:* The primary evaluation was done by utilizing test data points that simulate real-world options-pricing scenarios. They are shown in the table below.

TABLE II
INPUT DATA POINTS

S	K	r	sigma	T
25.0	25.0	0.05	0.20	1.0
30.0	25.0	0.03	0.25	1.0
25.0	30.0	0.05	0.20	1.0
20.0	20.0	0.01	0.10	2.0

These data points indicate four different types of call options (from top to bottom):

- **Option 1:** At-the-money call option (high sensitivity to volatility).
- **Option 2:** In-the-money call option (intrinsic value).
- **Option 3:** Out-the-money call option (speculative).
- **Option 4:** At-the-money call option (long duration, low volatility).

The call-option prices computed by the FPGA are shown below, along with their margins of error that were acquired through an external calculator.

TABLE III
COMPUTED VS. TRUE CALL PRICES

Test Case	Computed Call Price	True Call Price	Percent Error
1	2.550781	2.603	2.01%
2	6.649414	6.465	2.85%
3	0.856445	0.804	6.523%
4	1.319336	1.316	0.2535%

3) *Timing:* The primary purpose of this project is to compare the timing results of the ESP32 running Black-Scholes to the FPGA running the same formula. When the sample points in Table II are run through the ESP32, the following timing data is acquired:

TABLE IV
ESP32 TIMING TABLE

Test Case	Call Option Compute Time
1	668 us
2	377 us
3	341 us
4	352 us

Vivado has reported a synthesized critical *Data Path Delay* of **83.974ns**. This results in a total target clock frequency of roughly **11.9 MHz**. However, the FSM utilized in the design, as shown in Figure 2, takes six cycles between a *start* command is given and the *done* signal going high. This results in each computation taking **503.844 nanoseconds**, or a peak computation rate of **1.98 MHz**. Due to the deterministic nature of FPGA timing, each computation takes the same amount of time to complete. The table below showcases a timing comparison:

TABLE V
EXECUTION TIME AND SPEEDUP TABLE

Test Case	ESP32	FPGA	Speedup
1	668000 ns	503.844 ns	1325x
2	377000 ns	503.844 ns	748x
3	341000 ns	503.844 ns	677x
4	352000 ns	503.844 ns	699x

IV. DISCUSSIONS

The FPGA-accelerated Black-Scholes implementation has shown to provide a remarkable 600-1400x speedup compared to the microcontroller-based (ESP32) software implementation. This dramatic performance gain was primarily due to the FPGA's ability to perform application specific calculations with zero overhead. The FPGA's usage of hardware-accelerated calculations using precomputed lookup tables (LUTs) for complex mathematical functions was also a strong benefit. However, this speedup came at the cost of reduced accuracy, with the FPGA results exhibiting a 2-3% error margin compared to the software implementation. The primary sources of this inaccuracy were the use of fixed-point arithmetic (Q6.10 and 4.12 formats) and the limited resolution of the LUTs. Despite this trade-off, the speed gained makes the FPGA implementation highly suitable for high-frequency trading (HFT) applications, where low-latency calculations are critical.

A. Software and Physical Limitations

Despite the system's advancements, several limitations impact its performance in real-world HFT scenarios:

- 1) **API Latency:** While the external API integration with Alpha Vantage, Polygon, FMP, FINNHUB, TIINGO, and FRED is functional, there is a significant delay in retrieving pricing data. This latency undermines the system's ability to meet the nanosecond-level requirements of HFT, as API response times often range in the hundreds of milliseconds.
- 2) **API Rate Limits:** The number of API calls permitted per day by these services is far too low for live tracking, making continuous real-time data acquisition infeasible. For instance, free-tier API plans often limit users to a few hundred calls per day, which is insufficient for the high-frequency data needs of HFT.
- 3) **Wireless Communication Overhead:** The use of Wi-Fi for data fetching introduces additional latency compared to wired connections like direct Ethernet, which are standard in traditional HFT setups.
- 4) **FPGA Resource Constraints:** The Xilinx Artix-7 FPGA, while efficient, has limited resources for handling more complex models or multiple simultaneous computations, potentially restricting scalability.
- 5) **Power Efficiency Trade-offs:** Although designed to be low-power, the combined operation of the ESP32 and FPGA still consumes more energy than specialized ASICs, which may limit deployment in power-constrained environments.

B. Future Work - RTL

To address the RTL limitations of this project, several avenues for future work are proposed:

1) Enhanced Lookup Table Resolution

The current design uses lookup tables to accelerate nonlinear function evaluations in hardware. Specifically, a Q4.12 fixed-point format is used for the normal

cumulative distribution function (CDF), while Q6.10 is adopted for functions like square root and exponential. These functions are stored in precomputed tables with 513 entries, using hexadecimal representations for efficient ROM access. While this implementation offers speed and resource efficiency, its limited resolution can introduce noticeable quantization and interpolation errors—especially in regions where the function gradient changes rapidly. Future work will investigate expanding the resolution of these tables, either by increasing the number of entries or using non-uniform sampling techniques that concentrate precision in high-sensitivity regions. This will improve numerical fidelity and overall model accuracy, which is critical in financial applications where small errors can spread significantly.

2) **Leveraging DSP Cores**

The Arty A7 FPGA includes dedicated DSP slices optimized for high-throughput arithmetic operations. Currently underutilized, these resources can be exploited to accelerate core operations such as multiply accumulate (MAC), division approximations, and other signal processing kernels. Offloading arithmetic-intensive tasks to DSP blocks will reduce logic congestion and improve computation latency. However, they require advanced pipelining to operate efficiently.

3) **Pipelined Computational Architecture**

To maximize throughput and minimize computation delays, pipelining between the data processing and communication modules is critical. By dividing the computation into well-defined stages and processing multiple inputs concurrently, we can significantly reduce critical path lengths and support higher operating frequencies. This is particularly valuable for processing incoming tick data, performing model evaluations, and forwarding trading signals in a deterministic pipeline.

C. Future Work - Software and Hardware

To address the other software and physical limitations of this project, several avenues for future work are proposed:

- 1) **Optimized Data Acquisition:** Develop a caching mechanism or local data proxy to reduce dependency on external API calls, minimizing latency and bypassing rate limits. Alternatively, partnering with premium API providers offering higher rate limits and lower latency could enhance real-time performance.
- 2) **Wired Communication Options:** Incorporate an optional Ethernet module for the ESP32 to provide a low-latency wired connection when deployed in environments where Wi-Fi overhead is unacceptable.
- 3) **FPGA Scalability:** Upgrade to a more powerful FPGA, such as the Xilinx Kintex-7, to support additional computational tasks or more complex pricing models, improving scalability for larger HFT applications.
- 4) **Energy Optimization:** Explore power-efficient FPGA designs or integrate power-saving modes in the ESP32

to further reduce energy consumption, making the device more suitable for long-term edge deployments.

- 5) **Advanced Algorithms:** Extend the system to support additional HFT strategies, such as arbitrage or machine learning-based predictions, by integrating more sophisticated algorithms on the FPGA or leveraging external cloud resources for pre-processing.

V. DISTRIBUTION OF WORK

Each team member has made significant contributions to the project based on the assigned tasks, as outlined below.

- 1) **Aneesh:** Led the RTL logic design and verification, ensuring the FPGA implementation of the Black-Scholes model was accurate and optimized for low-latency performance.
- 2) **Dheeraj:** Assisted with RTL logic verification, supporting the development of the FPGA-based computation pipeline.
- 3) **Bharat:** Assisted with norm-cdf RTL module logic design and verification, supporting the development of the FPGA-based computation pipeline.
- 4) **Shraddha:** Wrote software for API integration and pulling financial data.
- 5) **Yijun:** Researched financial APIs and wrote code to acquire ESP32 benchmarking data.

These tasks were assigned flexibly and adapted during the development process to meet project needs, with each member contributing to the overall success of the benchmarking solution.

VI. CONCLUSIONS

This project successfully demonstrates the feasibility of a portable, low-power edge device for High-Frequency Trading (HFT) by showcasing the speedup between an ESP32 micro-controller and a Xilinx Artix-7 FPGA to implement the Black-Scholes option pricing model. Significant latency reduction through FPGA-based hardware acceleration is proven, and with the implementation of wireless data handling and SPI transfers, may address the geographical and cost barriers inherent in traditional HFT infrastructure. Despite challenges such as API latency and rate limits, such a device offers a promising solution for institutional traders, individual investors, and educational purposes, with potential applications in preliminary data processing, small-scale trading, and cross-market arbitrage. Future enhancements, including optimized data acquisition, wired communication options, and FPGA scalability, will further refine the system's performance, paving the way for its adoption in more demanding HFT environments. This work underscores the potential of edge computing to democratize access to advanced trading technologies while laying a foundation for continued innovation in the field.

REFERENCES

- [1] F. Black and M. Scholes, *The Pricing of Options and Corporate Liabilities*, Journal of Political Economy, vol. 81, no. 3, pp. 637–654, 1973.

- [2] A. Gautam, M. J. Khan, and M. Zahid, *Accelerating Black-Scholes Option Pricing Model Using FPGA: A Review*, *Microprocessors and Microsystems*, vol. 82, 103922, 2021. doi:10.1016/j.micpro.2021.103922.
- [3] Y. Zhang, H. Wang, and Z. Chen, *Ultra-Low Latency Financial Analytics Using FPGA-Accelerated Systems*, *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 16, no. 1, 2023. doi:10.1145/3571708.
- [4] L. Gao and K. Lin, *Edge Intelligence for Financial Trading: A Hardware/Software Co-Design Perspective*, *IEEE Access*, vol. 10, pp. 110530–110543, 2022. doi:10.1109/ACCESS.2022.3206590.
- [5] C. Leber, B. Geib, and H. Litz, *High Frequency Trading Acceleration Using FPGAs*, 2011 International Conference on Field Programmable Logic and Applications (FPL), pp. 317–322. doi:10.1109/FPL.2011.64.
- [6] K. A. Aly, *Introducing FPGA-Based Acceleration for High-Frequency Trading*, *EE Times*, 2014. Available: <https://www.eetimes.com/introducing-fpga-based-acceleration-for-high-frequency-trading/>
- [7] Vemeko Technologies, *How to Use FPGAs for High-Frequency Trading (HFT) Acceleration?*, 2025. Available: <https://www.vemeko.com/blog/67121.html>
- [8] Velvetech LLC, *FPGA-Based Hardware Acceleration for High-Frequency Trading Systems*, 2023. Available: <https://www.velvetech.com/fpga-based-hardware-acceleration/>
- [9] Espressif Systems, *ESP32 Technical Reference Manual*, Version 4.3, 2023. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [10] Digilent Inc., *Arty A7 Reference Manual*, 2022. Available: <https://digilent.com/reference/programmable-logic/arty-a7/start>
- [11] AMD, *Vivado Design Suite User Guide: Getting Started (UG910)*, Version 2024.2, 2024. Available: <https://docs.amd.com/r/en-US/ug910-vivado-getting-started>